

General Training for Programming Competitions & Coding Interviews

Winter Semester 2024/2025

Graph Algorithms

Markus Fischer, Paul G. Rump, Christoph Staudt

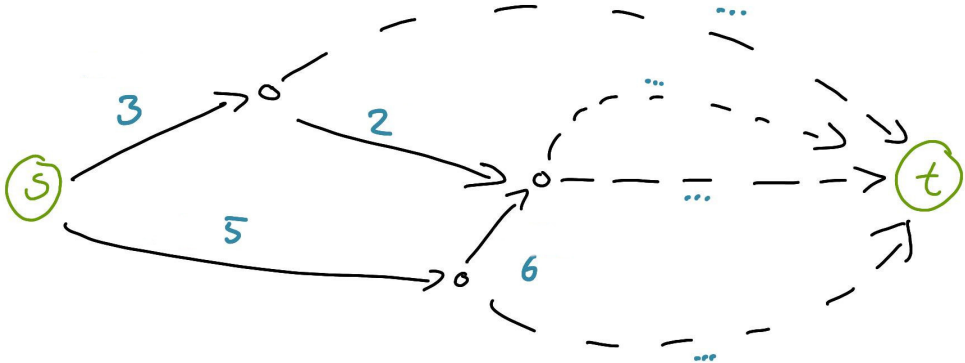
Faculty of Mathematics and Computer Science



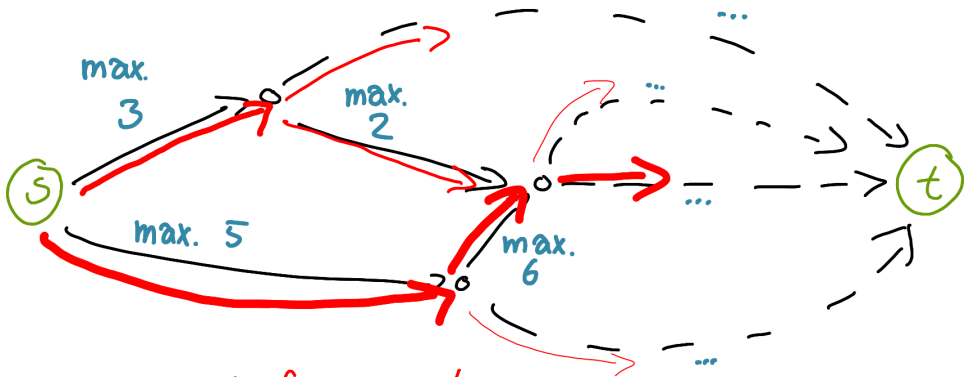
Maximum Flow & Minimum Cut

MAXIMUM FLOW & MINIMUM CUT

Weighted directed graph

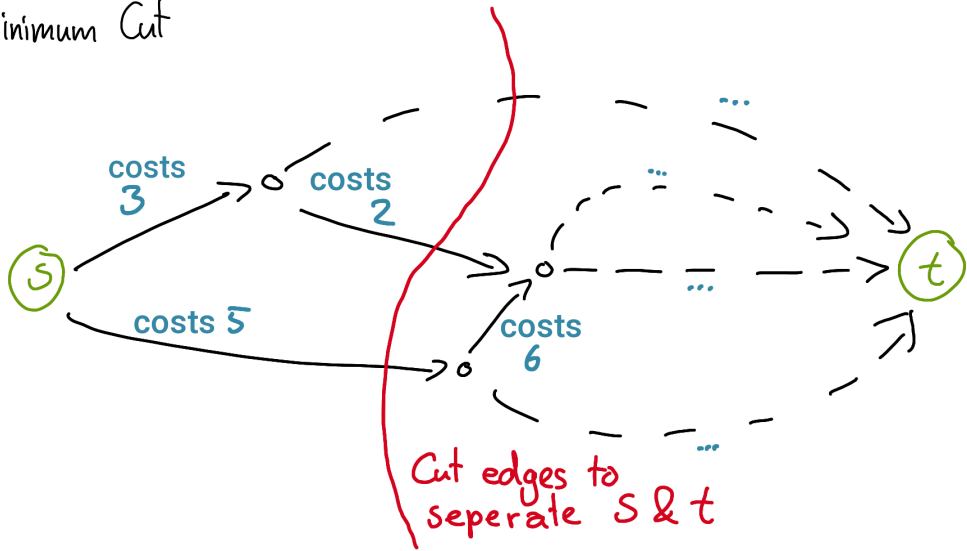


Maximum Flow



maximize total flow $s \rightarrow t$

Minimum Cut



The Maximum-Flow Problem

Task:

- Given a graph $G = (V, E)$ with edge capacities $c : E \rightarrow \mathbb{R}^+$ (called *flow network*), and
- a pair of nodes $s, t \in V$ (called *source* and *sink*).
- Find a *flow*, that is a mapping $f : E \rightarrow \mathbb{R}^+$ with:
 - Capacity Constraints: $\forall e \in E : f(e) \leq c(e)$
 - Conservation of flow:

$$\forall v \in V \setminus \{s, t\} : \sum_{u \in V : (u, v) \in E} f((u, v)) = \sum_{w \in V : (v, w) \in E} f((v, w))$$

- The *size* of the flow is the amount of flow that leaves s , that is:

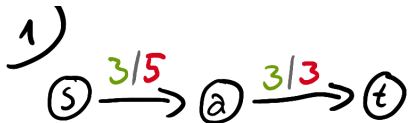
$$F = \sum_{v \in V : (s, v) \in E} f((s, v))$$

This is the value we need to maximize.

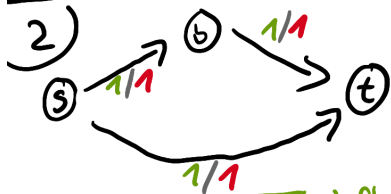
The Maximum-Flow Problem

- Note that the flow constraints imply that the amount of flow that leaves s is equal to the amount of flow that goes into t .
- Often, the graph of a max-flow-problem is a directed graph, and the algorithms expect a directed graph as input. However, most algorithms work also for undirected graphs.

Examples of Graphs and their Maximum Flow

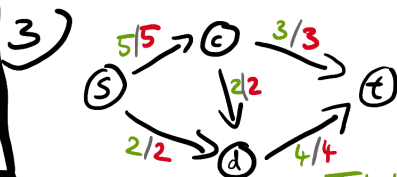


Total flow : 3

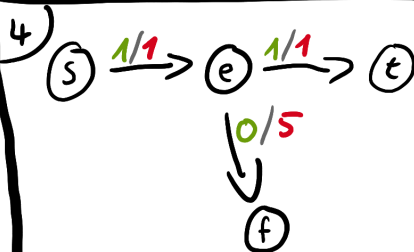


Total flow : 2

Flow of the edge
Capacity of the edge



Total flow: 7



Total flow : 1

The Minimum-Cut Problem

Definition: Cut

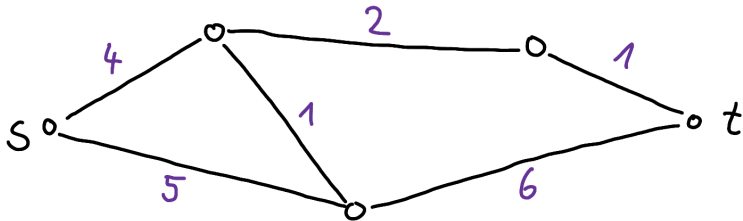
- Given a graph $G = (V, E)$ with edge weights $c : E \rightarrow \mathbb{R}^+$, and
- a pair of nodes $s, t \in V$.
- A partition of the nodes of G into $S, T \subset V$ with $s \in S, t \in T$ is called a *s-t-cut*, and the *cut-set* is $C(S, T) = \{e = (u, v) \in E \mid u \in S, v \in T\}$

Definition: Minimum Cut:

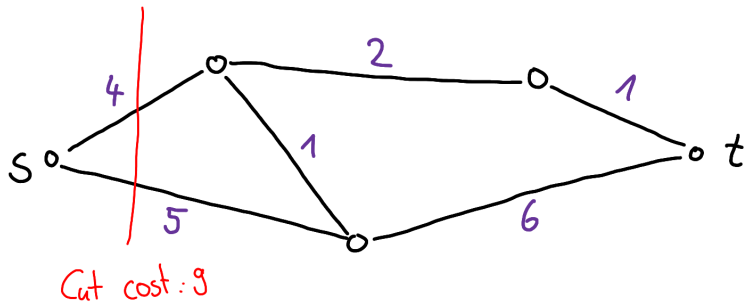
- Given $G = (V, E)$, $c : E \rightarrow \mathbb{R}^+$, and $s, t \in V$ as above.
- Find a *s-t-cut* (S, T) such that $\sum_{e \in C(S, T)} c(e) \leq \sum_{e \in C(S', T')} c(e)$ for all other *s-t-cuts* (S', T') .

That is: a cut where the *sum of the edges of the cut-set is minimal*.

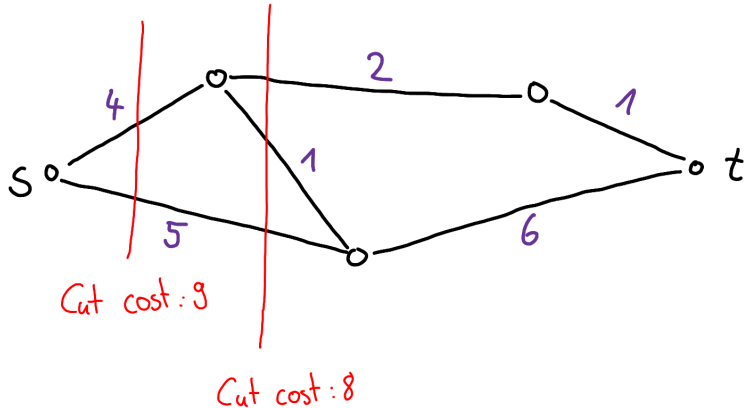
Examples of (Minimum) Graph Cuts



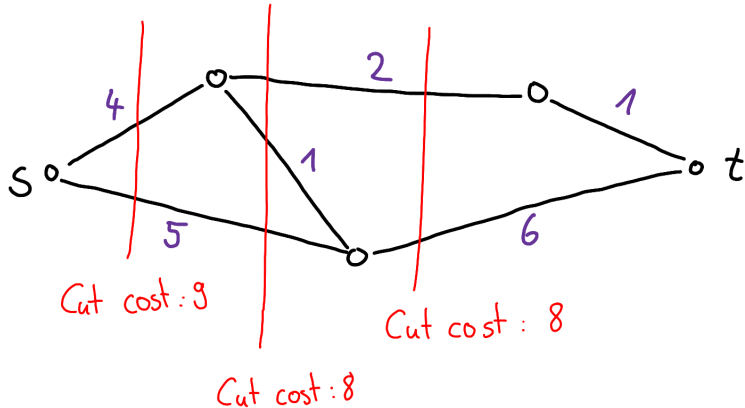
Examples of (Minimum) Graph Cuts



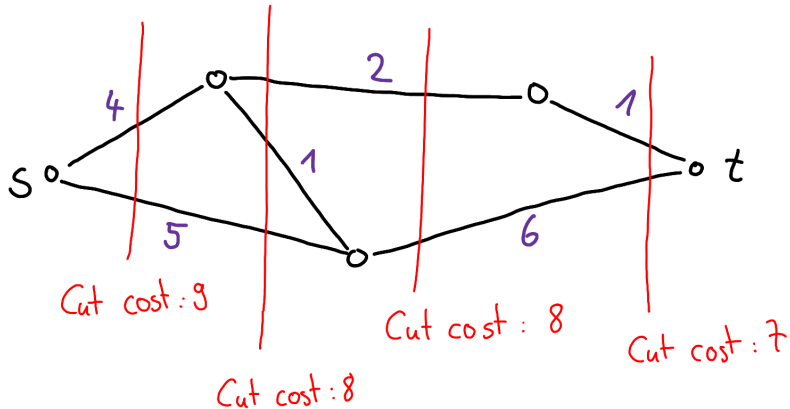
Examples of (Minimum) Graph Cuts



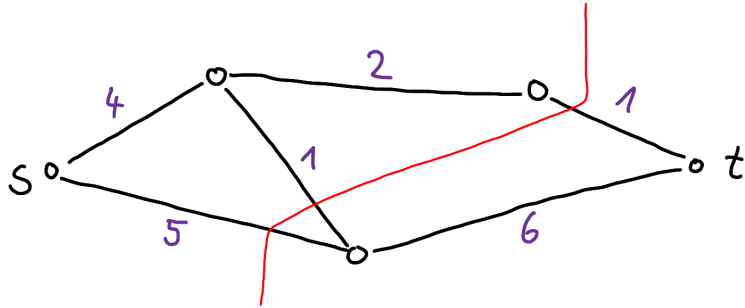
Examples of (Minimum) Graph Cuts



Examples of (Minimum) Graph Cuts

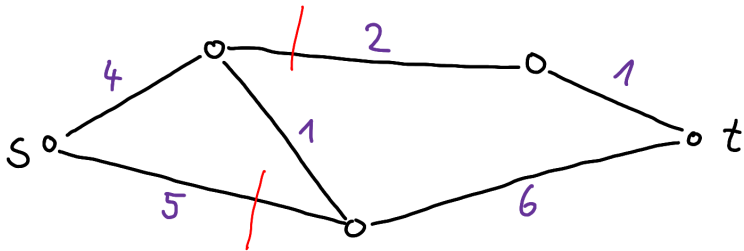


Examples of (Minimum) Graph Cuts



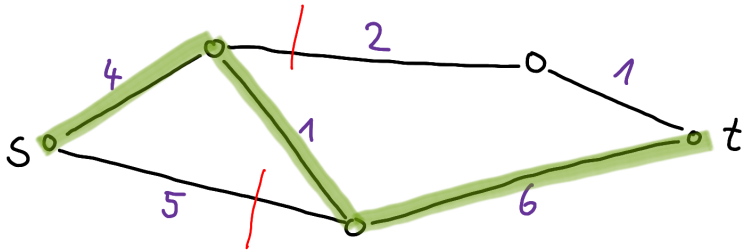
Cut cost: 7

Examples of (Minimum) Graph Cuts



not an s - t -cut!

Examples of (Minimum) Graph Cuts



not an s-t-cut!

MaxFlow = MinCut

The *Max-Flow-Min-Cut-Theorem* states that in a flow network the *maximum amount of flow* passing from the source to the sink is *equal* to the *total weight of the edges in a minimum cut*.

- For any s - t -cut, an s - t flow is restricted by the edges of the cut
 $\Rightarrow \text{MaxFlow} \leq \text{MinCut}$.
- To prove the Max-Flow-Min-Cut-Theorem, one has to show that also $\text{MinCut} \leq \text{MaxFlow}$ holds.

You can easily find a proof online if you are interested.

Extensions

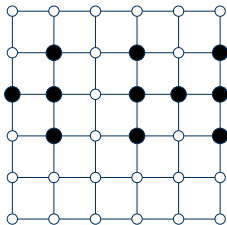
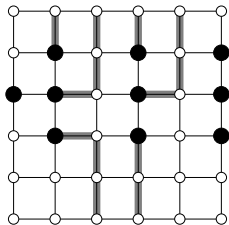
- How can we extend the maximum flow problem to several sources and sinks?
- How can we add capacities to the nodes?

Escape Problem

Consider an $n \times n$ grid where the intersection points are labeled $(i, j) \in \{1, \dots, n\} \times \{1, \dots, n\}$, denoting that the point is in row i and column j . Each intersection point is connected to up to four neighbors, depending on whether it is on the edge or within the grid.

Given $m \leq n^2$ distinct starting points $(x_1, y_1), \dots, (x_m, y_m)$ in the grid, the escape problem asks whether there exist m node-disjoint paths to the grid edge starting from these m starting points.

Below are two examples with $n = 6$ and $m = 10$ (left, solvable) and $m = 11$ (right, unsolvable):



Algorithm by Edmonds and Karp

- Basic idea: starting from an empty flow, find any valid (s,t) -flow with as few edges as possible. Add this flow to the total flow and adjust the remaining capacities accordingly. Iterate until no additional flow is found.
- “Adjusting the capacities” means: *reducing* the capacities in direction of the flow, and *increasing* the capacities in opposite direction.
- The idea of this algorithm is by Ford and Fulkerson, but their algorithm has suboptimal behavior in special cases.
- The Edmonds-Karp-algorithm builds on the same idea.

Code for Edmonds-Karp algorithm

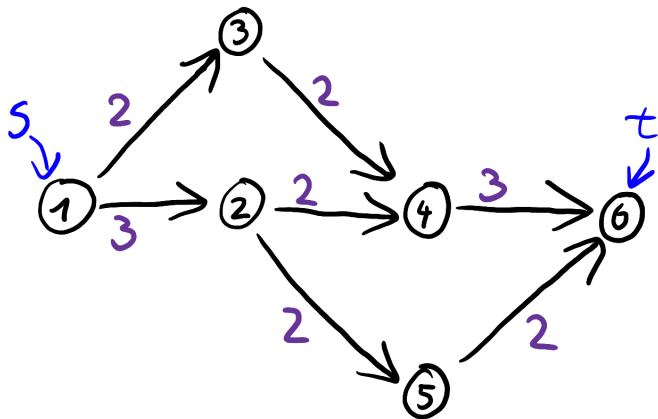
See file `edmonds-karp.cpp`

Algorithm by Edmonds and Karp

The algorithm has time complexity $\mathcal{O}(|V| \cdot |E|^2)$:

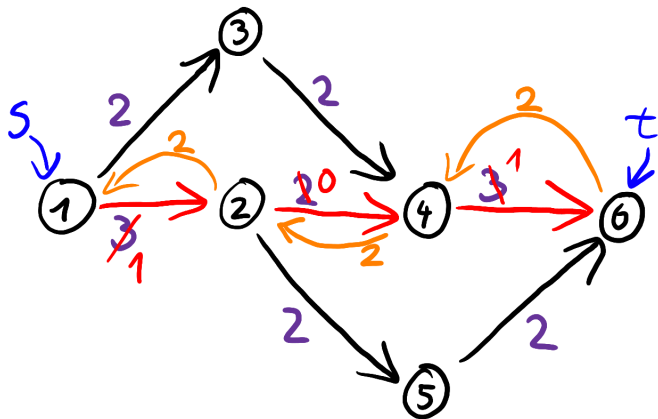
- It takes $\mathcal{O}(|E|)$ time to find an augmenting path (BFS).
- Each augmenting path has length at most $|V|$.
- Each augmenting path fully saturates at least one edge, therefore the algorithm does at most $|E|$ -many iterations.

Example of Edmonds-Karp-algorithm



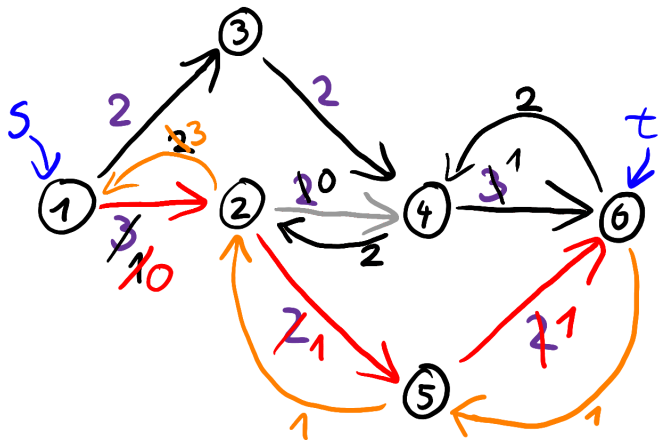
All flows so far:

Example of Edmonds-Karp-algorithm



All flows so far:
 $1 \rightarrow 2 \rightarrow 4 \rightarrow 6 : 2$

Example of Edmonds-Karp-algorithm

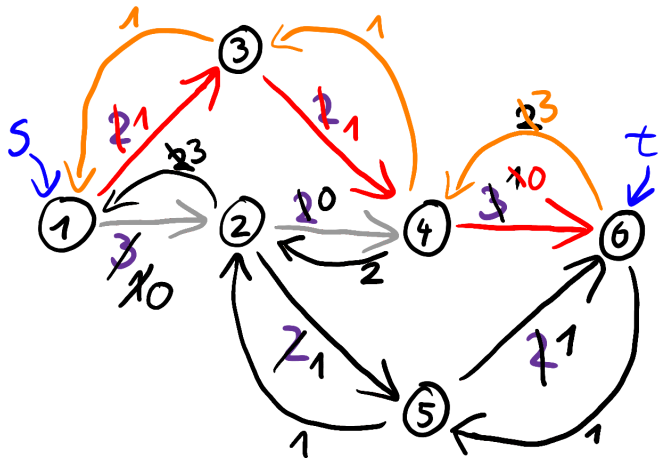


All flows so far:

$1 \rightarrow 2 \rightarrow 4 \rightarrow 6 : 2$

$1 \rightarrow 2 \rightarrow 5 \rightarrow 6 : 1$

Example of Edmonds-Karp-algorithm



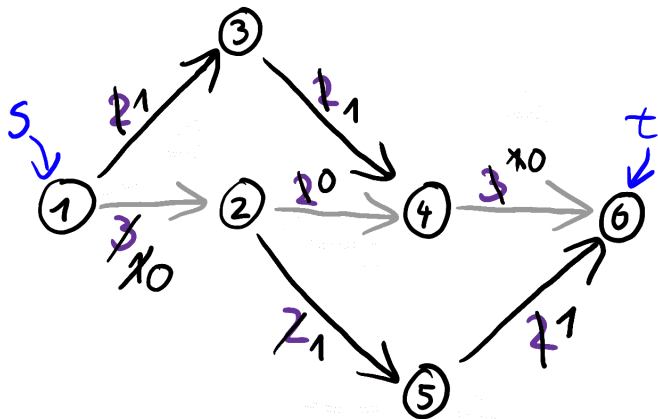
All flows so far:

$1 \rightarrow 2 \rightarrow 4 \rightarrow 6 : 2$

$1 \rightarrow 2 \rightarrow 5 \rightarrow 6 : 1$

$1 \rightarrow 3 \rightarrow 4 \rightarrow 6 : 1$

Example of Edmonds-Karp-algorithm



All flows so far:

$1 \rightarrow 2 \rightarrow 4 \rightarrow 6 : 2$

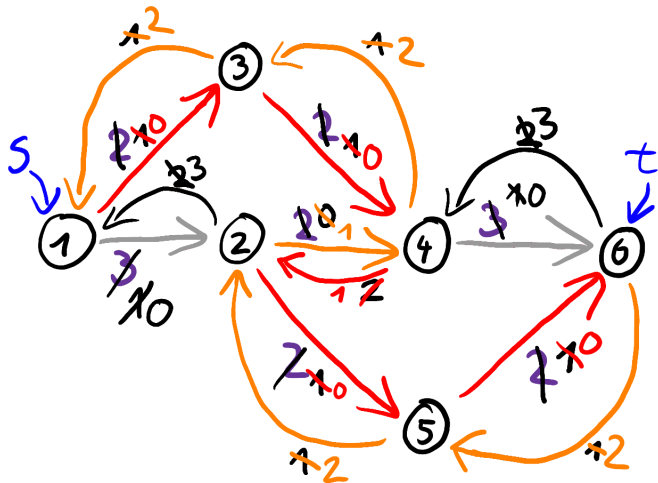
$1 \rightarrow 2 \rightarrow 5 \rightarrow 6 : 1$

$1 \rightarrow 3 \rightarrow 4 \rightarrow 6 : 1$

$\Sigma \quad 4$

But is this the
maximum flow?

Example of Edmonds-Karp-algorithm



All flows so far:

$1 \rightarrow 2 \rightarrow 4 \rightarrow 6 : 2$

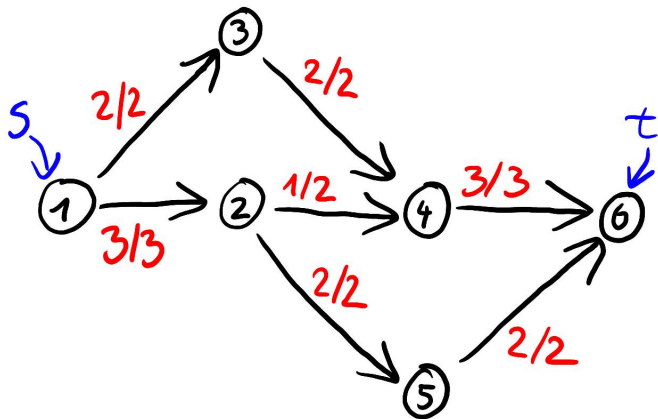
$1 \rightarrow 2 \rightarrow 5 \rightarrow 6 : 1$

$1 \rightarrow 3 \rightarrow 4 \rightarrow 6 : 1$

$1 \rightarrow 3 \rightarrow 4$

$\rightarrow 5 \rightarrow 6 : 1$

Example of Edmonds-Karp-algorithm



All flows so far:

$1 \rightarrow 2 \rightarrow 4 \rightarrow 6 : 2$

$1 \rightarrow 2 \rightarrow 5 \rightarrow 6 : 1$

$1 \rightarrow 3 \rightarrow 4 \rightarrow 6 : 1$

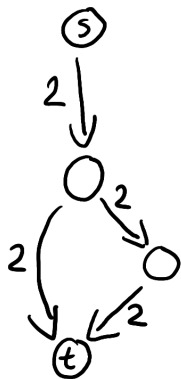
$1 \rightarrow 3 \rightarrow 4$

$\rightarrow 5 \rightarrow 6 : 1$

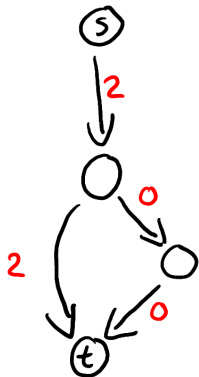
$\Sigma 5$

$\rightarrow$ This is the maximum!

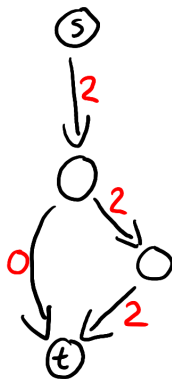
Flows are not unique!



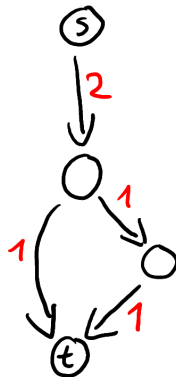
A flow network



A max flow



Another max flow



Another max flow

Dinic's algorithm

- Very similar to Edmonds-Karp, but often faster in practice.
- During the BFS we build a *level graph* that is used to find augmenting paths. In the level graph we only keep edges that lead to nodes further away from the source.
- Then we find augmenting paths in the level graph using a DFS, until there is no path from source to sink.
- The resulting flow is called a *blocking flow* and is added to the total flow in the main loop.

Exercise

Exercise

- The exercise for this week is again a programming contest.
- This time, you will work on old problems from the ICPC (however these won't be problems from a world final).
- The website <https://onlinejudge.org/> hosts a contest based on these old problems. You can upload your solution, it gets tested and you get a global rank based on the runtime of your code.
- While they support multiple programming languages (not as many as codingame), I *strongly suggest to use C++*, because it's the fastest in general.

Exercise - Preparation

1. Create an account at <https://onlinejudge.org/> (Optional).
2. Work on the following two problems:
 - Problem 100: The $3n+1$ -problem
You should have no big problems in solving this. This is a simple problem to learn how the contest works. However, you can still optimize a lot if you like to.
 - Problem 459: Graph Connectivity
Remember what you learned about graph algorithms.
3. For both problems, construct additional interesting test cases (at least 3 per problem).
4. Upload your code (and test cases) to your git repository.

Problem settings are also included on the next two pages.

Problems in Computer Science are often classified as belonging to a certain class of problems (e.g., NP, Unsolvable, Recursive). In this problem you will be analyzing a property of an algorithm whose classification is not known for all possible inputs.

Consider the following algorithm:

1. input n
2. print n
3. if $n = 1$ then STOP
4. if n is odd then $n \leftarrow 3n + 1$
5. else $n \leftarrow n/2$
6. GOTO 2

Given the input 22, the following sequence of numbers will be printed

22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1

It is conjectured that the algorithm above will terminate (when a 1 is printed) for any integral input value. Despite the simplicity of the algorithm, it is unknown whether this conjecture is true. It has been verified, however, for all integers n such that $0 < n < 1,000,000$ (and, in fact, for many more numbers than this.)

Given an input n , it is possible to determine the number of numbers printed before and including the 1 is printed. For a given n this is called the *cycle-length* of n . In the example above, the cycle length of 22 is 16.

For any two numbers i and j you are to determine the maximum cycle length over all numbers between and including both i and j .

Input

The input will consist of a series of pairs of integers i and j , one pair of integers per line. All integers will be less than 10,000 and greater than 0.

You should process all pairs of integers and for each pair determine the maximum cycle length over all integers between and including i and j .

You can assume that no operation overflows a 32-bit integer.

Output

For each pair of input integers i and j you should output i , j , and the maximum cycle length for integers between and including i and j . These three numbers should be separated by at least one space with all three numbers on one line and with one line of output for each line of input. The integers i and j must appear in the output in the same order in which they appeared in the input and should be followed by the maximum cycle length (on the same line).

Sample Input

```
1 10
100 200
201 210
900 1000
```

Sample Output

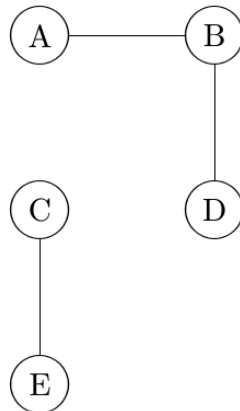
```
1 10 20
100 200 125
201 210 89
900 1000 174
```

Consider a graph G formed from a large number of nodes connected by edges. G is said to be *connected* if a path can be found in 0 or more steps between any pair of nodes in G . For example, the graph below is not connected because there is no path from A to C.

This graph contains, however, a number of subgraphs that are connected, one for each of the following sets of nodes: $\{A\}$, $\{B\}$, $\{C\}$, $\{D\}$, $\{E\}$, $\{A,B\}$, $\{B,D\}$, $\{C,E\}$, $\{A,B,D\}$

A connected subgraph is *maximal* if there are no nodes and edges in the original graph that could be added to the subgraph and still leave it connected. There are two maximal connected subgraphs above, one associated with the nodes $\{A,B,D\}$ and the other with the nodes $\{C,E\}$.

Write a program to determine the number of maximal connected subgraphs of a given graph.



Input

The input begins with a single positive integer on a line by itself indicating the number of the cases following, each of them as described below. This line is followed by a blank line, and there is also a blank line between two consecutive inputs.

The first line of each input set contains a single uppercase alphabetic character. This character represents the largest node name in the graph. Each successive line contains a pair of uppercase alphabetic characters denoting an edge in the graph.

The sample input section contains a possible input set for the graph pictured above.

Input is terminated by a blank line.

Output

For each test case, write in the output the number of maximal connected subgraphs. The outputs of two consecutive cases will be separated by a blank line.

Sample Input

```
1

E
AB
CE
DB
EC
```

Sample Output

```
2
```

Template for input and output

template_OJ_100.cpp

./code/template_OJ_100.cpp

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int a, b;
7     while (cin >> a >> b)
8     {
9
10         /*
11         compute solution
12         */
13
14         int solution = 0;
15
16         cout << a << " " << b << " " << solution << "\n";
17     }
18     return 0;
19 }
```

Exam Questions

Exam Questions

1. Research the difference between the algorithms by Edmonds-Karp and Ford-Fulkerson. Give an example where the Ford-Fulkerson-algorithm might take significantly longer than Edmonds-Karp.
2. Describe your algorithm that solves the Problem 100 (" $3n+1$ -problem"): what algorithmic techniques did you use?
3. Which algorithm do you use for Problem 459 ("Graph Connectivity"), and why?